

Projektdokumentation

Biofeedback-Visualisierung: Herzfrequenz als Flamme

Ein Prototyp in Unity

Name: Alexander Höher

Bearbeitungszeitraum: 31.08.2026 bis 11.09.2026

Inhalt

1. Projektbeschreibung	2
1.1 Was ist der Auftrag?	2
1.2 Für Wen?	2
1.3 Ziel des Projekts	2
2. Anforderungen	2
2.1 Anforderungen	2
2.2 Parameter, Einheiten und Normbereiche	3
2.3 Sensordaten.....	3
3. Projektumfeld.....	3
3.1 Hardware	3
3.2 Software, Plug-Ins und Assets	3
4. Schnittstellen	3
4.1 Benutzeroberfläche (UI).....	3
4.2 Bedienung der Anwendung	3
4.3 Daten-Schnittstelle (Datenbank, CSV, JSON)	3
5. Durchführung.....	3
5.1 Vorgehensweise im Überblick	3
5.2 Designevolution: von analog zu digital	4
5.3 Einarbeitung in Unity und Arbeit mit KI.....	4
5.4 Skripte, Hierarchie und Verknüpfung mit 3D-Objekten	4
5.6 Herausforderungen und Lösungen.....	5
5.7 Begründung zentraler Entscheidungen	6
5.8 Future Work	6
5.9 Fazit	6

1. Projektbeschreibung

1.1 Was ist der Auftrag?

Im Rahmen der Projektarbeit soll ein 3D-Modell entstehen, das als Biofeedback dient. Biofeedback bezeichnet eine Technik, bei der körperliche Prozesse – hier die Herzfrequenz – gemessen und der Person in Echtzeit zurückgemeldet werden, damit sie unbewusste Körperfunktionen bewusst wahrnehmen und beeinflussen kann. Umgesetzt wird eine Anwendung in der Game-Engine Unity, in der eine Flamme ihr Aussehen und die Herzfrequenz anpasst.

1.2 Für Wen?

Die Anwendung richtet sich an Menschen, die ihren Anspannungs- beziehungsweise Entspannungszustand intuitiv erleben möchten, ohne nackte Zahlen interpretieren zu müssen. Statt „82 bpm“ zu lesen, sehen sie eine ruhige oder eine unruhige Flamme. Diese Dokumentation wiederum richtet sich an Junior-Entwickler, die den Prototyp verstehen, warten oder weiterentwickeln.

1.3 Ziel des Projekts

Ziel ist ein lauffähiger Prototyp, in dem eine simulierte Herzfrequenz die Flamme über mehrere Kanäle steuert: Größe und Fülle, Flackergeschwindigkeit, Farbe von warmem Gelb bei Ruhe bis Rot bei Stress, Lichtintensität sowie einen Herzschlag-Ton im Puls-Takt. Ergänzend erscheint bei hohem Puls eine Atem-Anleitung, die den Feedback-Kreislauf schließt: Der Nutzer atmet ruhiger, der Puls sinkt, die Flamme wird ruhiger und gelber.

2. Anforderungen

2.1 Anforderungen

Funktional muss die Anwendung Folgendes leisten. Die Flamme ändert ihre Fülle über die Emission und ihre Größe abhängig von der Herzfrequenz, und ihre Flackergeschwindigkeit steigt mit dem Puls. Die Flammenfarbe wechselt fließend von Gelb im ruhigen Zustand zu Rot bei Stress. Eine Lichtquelle atmet in Intensität und Takt mit der Flamme mit, und ein Herzschlag-Ton wird im Takt der aktuellen Herzfrequenz ausgelöst. Die aktuelle Herzfrequenz wird zusätzlich als Text in der Form „xx bpm“ angezeigt. Der Puls wird automatisch und lebendig simuliert, lässt sich zur Vorführung aber auch per Tastatur gezielt erhöhen oder senken. Ab einer Schwelle erscheint eine ein- und ausblendende Atem-Anleitung. Schließlich ist der Blick steuerbar - am Bildschirm per Maus, in der VR-Ausbaustufe automatisch per Kopfbewegung..

2.2 Parameter, Einheiten und Normbereiche

Die einzige physiologische Größe ist die Herzfrequenz in bpm. Der Arbeitsbereich der Anwendung reicht von 50 bis 120 bpm; zum Vergleich liegt der Ruhepuls eines Erwachsenen bei etwa 60 bis 80 bpm. Die automatische Simulation schwankt im Ruhemodus in einem frei wählbaren Bereich von rund 55 bis 75 bpm. Der Atem-Hinweis blendet sich ab 95 bpm ein und erst wieder unter 85 bpm aus - die Lücke dazwischen ist eine Pufferzone gegen Flackern. Das Herzschlag-Intervall ergibt sich aus 60 geteilt durch die Herzfrequenz; bei 60 bpm entspricht das genau einer Sekunde pro Schlag..

2.3 Sensordaten

Im Prototyp werden die Herzfrequenzwerte in der Anwendung simuliert, nicht gemessen. Der Wert liegt als einzelne Gleitkommazahl in bpm vor und wird an einer zentralen Stelle im Code erzeugt. Diese Stelle ist bewusst gekapselt, sodass später reale Sensordaten - etwa von einem Pulssensor über einen Raspberry Pi - an genau dieser Stelle eingespeist werden könnten, ohne die Visualisierung zu ändern. Mehr dazu im Abschnitt Future Work.

3. Projektumfeld

3.1 Hardware

Entwickelt wurde auf einem HP EliteBook Windows 11. Als Eingabe beim Desktop-Test dienen Maus und Tastatur. Nicht Teil des Prototyps, aber als Ausblick vorgesehen ist ein Raspberry Pi mit einem Pulssensor.

3.2 Software, Plug-Ins und Assets

Als Engine kommt Unity in der Version 6000.5.10f1 mit der Universal Render Pipeline zum Einsatz. Programmiert wurde in C# in der Entwicklungsumgebung Visual Studio. Für die Eingabe ist das Unity Input System aktiv und die Textanzeigen für die Herzfrequenz und die Atem-Anleitung nutzen TextMeshPro. Die Versionsverwaltung erfolgt über Git.

Drei kostenlose Assets aus dem Unity Asset Store kommen zum Einsatz, jeweils unter der Standard Unity Asset Store EULA als Extension Asset: die Flamme „Free Fire VFX – URP“ von Vefects, die Burg „My Fantasy Fortress“ von moi_lolita und der Sternenhimmel „Real Stars Skybox Lite“ von Geoff Dallimore. Hinzu kommt ein Herzschlag-Sound aus freesound.org.

4. Schnittstellen

4.1 Benutzeroberfläche (UI)

Die Benutzeroberfläche ist bewusst minimal, um den Fokus auf der Flamme zu halten. Sie besteht aus zwei TextMeshPro-Elementen: der bpm-Anzeige, die die aktuelle Herzfrequenz als Zahl zeigt, und dem Atem-Hinweis „Einatmen ... / Ausatmen ...“, der nur bei hohem Puls erscheint. Aktuell liegt die UI im Screen-Space, klebt also fest am Bildschirm.

4.2 Bedienung der Anwendung

Die Bedienung ist auf wenige Eingaben beschränkt. Mit der Pfeiltaste nach oben lässt sich die Herzfrequenz erhöhen, wodurch die Flamme größer, röter und schneller wird; die Pfeiltaste nach unten senkt sie wieder und beruhigt die Flamme. Die Maus steuert im Desktop-Test die Blickrichtung, in der VR-Ausbaustufe übernimmt das die Kopfbewegung automatisch. Ohne jede Eingabe schwankt der Puls von selbst weich im Ruhebereich.

4.3 Daten-Schnittstelle (Datenbank, CSV, JSON)

Der Prototyp verwendet bewusst keine Datenbank und keine Persistenz - es werden keine Daten gespeichert oder exportiert. Die interne Schnittstelle ist die zentrale Variable für die Herzfrequenz im FlameController. Eine künftige Anbindung könnte an dieser Stelle andocken: Ein externer Sensor am Raspberry Pi würde bpm-Werte, etwa als kleine JSON-Nachricht über das Netzwerk, senden, die die Anwendung empfängt und anstelle des simulierten Werts verwendet. Ebenso wäre ein späterer CSV-Export des Puls-Verlaufs zu Auswertungszwecken denkbar. Alle wichtigen Konfigurationswerte sind zudem ohne Code-Änderung direkt im Unity-Inspector einstellbar.

5. Durchführung

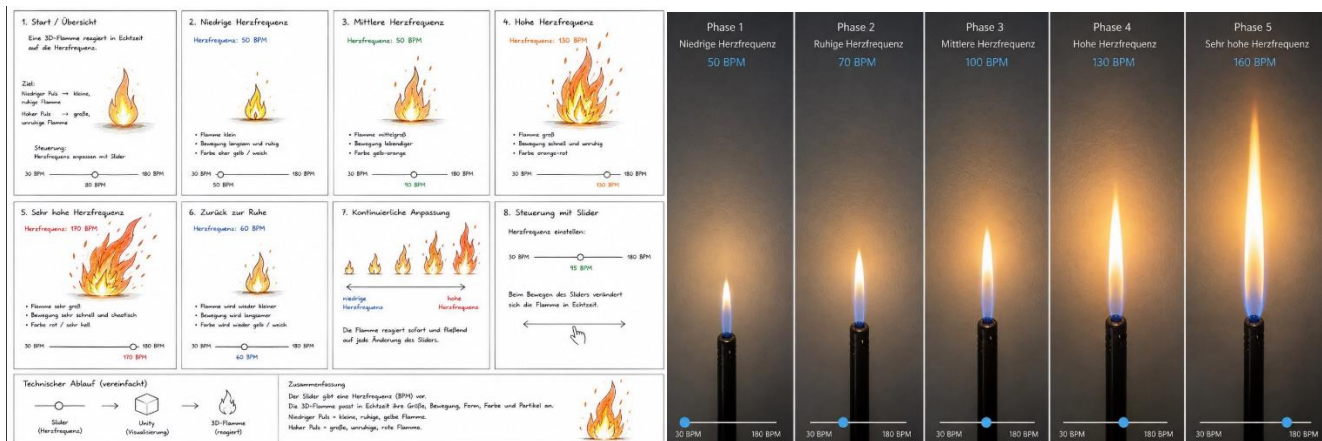
5.1 Vorgehensweise im Überblick

Ich entwickelte in klar abgegrenzten Phasen. Zunächst kümmerte ich mich um den analoge Prototypen und das Storyboard, danach um den digitale Prototyp in Unity. Dort habe ich zuerst die Umgebung aufgebaut - ein dunkler Raum mit Sternenhimmel und einer Burg -, anschließend das Flammen-Asset eingebunden und dessen Steuerung entwickelt, zunächst über einen Slider, später über eine automatische Simulation mit ergänzender Tastatursteuerung. Darauf folgten die Ausgabekanäle- Größe, Flackern, Farbe, Licht, Herzschlag-Ton und die bpm-Anzeige - sowie als aktives Feedback die Atem-Anleitung.

5.2 Designevolution: von analog zu digital

Vor der ersten Zeile Code stellte ich mir die Frage, wie sich eine Flamme überhaupt „ruhig“ oder „gestresst“ verhält. Als physischer Prototyp verwendete ich eine echte Feuerzeugflamme, Beobachtet wurde, wie sie bei ruhiger Luft klein und gleichmäßig brennt und bei Bewegung oder Zug größer wird und hektisch flackert. Diese Beobachtung wurde direkt zur Design-Regel für die digitale Flamme: ruhig bedeutet klein, gelb und langsam, Stress bedeutet groß, rot und schnell.

Anschließend hielt ein Storyboard den geplanten Nutzungsablauf fest: Der Nutzer sieht eine ruhige Flamme, seine Anspannung steigt, die Flamme wird größer und röter, der Nutzer atmet ruhig, und die Flamme beruhigt sich wieder. Das Storyboard machte frühzeitig klar, welche Bausteine überhaupt nötig sind.



5.3 Einarbeitung in Unity und Arbeit mit KI

Zu Projektbeginn beherrschte ich keine Unity-Kenntnisse. Die Engine, der Aufbau des Editors mit Hierarchy, Inspector sowie Scene- und Game-Ansicht und die Sprache C# musste ich zunächst erlernen. Die Einarbeitung erfolgte begleitend zur Umsetzung, also direkt am eigenen Projekt statt an abstrakten Übungen.

Unterstützend habe ich mit dem KI-Assistenten Claude gearbeitet. Für mich hat sich ein iterativer, prüfender Stil bewährt: Aufgaben klein formulieren, den vorgeschlagenen Code einsetzen, das Ergebnis im Editor testen und beobachtete Probleme oder Fehlermeldungen wortgetreu zurückspielen. Die KI lieferte Code, Erklärungen der Unity-Begriffe und gezielte Fehlersuche; die inhaltlichen Entscheidungen, das Testen und die Bewertung im Editor blieben beim mir.

5.4 Skripte, Hierarchie und Verknüpfung mit 3D-Objekten

Das Herzstück ist ein einziges C#-Skript, der FlameController. Es hält den simulierten Puls, wandelt ihn in einen Anteil zwischen 0 für ruhig und 1 für Stress um und steuert damit alle Ausgabekanäle. Die zu steuernden Objekte werden nicht im Code gesucht, sondern im Inspector zugewiesen - das ist robuster und für Einsteiger nachvollziehbarer. Das Skript liegt auf dem Feuer-Objekt und stellt dafür öffentliche Felder bereit.

Über das Feld für die Flame Systems werden ausschließlich die beiden Kern-Partikelsysteme der Flamme zugewiesen - flames und flames secondary -, während Funken und Asche bewusst unangetastet bleiben. Das Feld Bpm Text verweist auf die TextMeshPro-Anzeige der Herzfrequenz, das Feld Breath Text auf das Textfeld der Atem-Anleitung. Das Feld Flame Light nimmt das Point Light auf, das die Umgebung beleuchtet, und das Feld Heartbeat die AudioSource mit dem Herzschlag-Clip. Zwei Farbfelder legen die Ruhe- und die Stressfarbe fest, also Gelb und Rot. Weitere Felder wie Calm Min und Calm Max sowie Smoothing steuern das Verhalten der automatischen Puls-Simulation, und die Felder Breath On, Breath Off und Fade Speed regeln die Schwellen und das Ein- und Ausblenden der Atem-Anleitung..

Damit die fein abgestimmte Optik des Assets erhalten bleibt, werden dessen Ausgangswerte beim Start gemerkt und der Puls-Faktor darauf multipliziert, statt die Werte hart zu überschreiben:

```
void Start()
{
    // Für jedes Flammen-System die Original-Werte merken.
    baseRate = new float[flameSystems.Length];
    baseSize = new float[flameSystems.Length];
    for (int i = 0; i < flameSystems.Length; i++)
    {
        baseRate[i] = flameSystems[i].emission.rateOverTimeMultiplier;
        baseSize[i] = flameSystems[i].main.startSizeMultiplier;
    }
}
```

Aus dem Puls wird mit Inverselerp ein Anteil t zwischen 0 und 1 berechnet; Lerp mischt damit die Faktoren und die Farbe:

```
// t = Anteil zwischen 0 (ruhig) und 1 (Stress).
float t = Mathf.InverseLerp(minBpm, maxBpm, currentBpm);

// Flacker-Wert: eine schwingende Zahl um 1 herum, schneller bei Stress.
float flickerSpeed = Mathf.Lerp(3f, 15f, t);
float flicker = 1f + Mathf.Sin(Time.time * flickerSpeed) * Mathf.Lerp(0.05f, 0.2f, t);

// Faktoren für Menge und Größe (bei Ruhe ~ Original, bei Stress deutlich mehr).
float rateFactor = Mathf.Lerp(1.0f, 4.5f, t) * flicker;
float sizeFactor = Mathf.Lerp(0.7f, 2.4f, t) * flicker;

// Farbe zwischen Gelb und Rot mischen.
Color currentColor = Color.Lerp(calmColor, stressColor, t);
```

Der Herzschlag folgt dem Puls-Takt, denn 60 geteilt durch die Herzfrequenz ergibt die Sekunden zwischen zwei Schlägen:

```
// 60 / bpm = Sekunden zwischen zwei Schlägen (z. B. 60 bpm -> 1 Sekunde).
float interval = 60f / currentBpm;

beatTimer += Time.deltaTime;
if (beatTimer >= interval)
{
    heartbeat.Play();
    beatTimer = 0f;
}
```

Der Atem-Hinweis erscheint ab 95 bpm und verschwindet erst unter 85 bpm, sodass er nicht flackert, wenn der Puls um die Schwelle pendelt:

```
// Pufferzone: an ab breathOn, aus erst unter breathOff (kein Flackern an der Grenze).
if (currentBpm >= breathOn) breathingActive = true;
else if (currentBpm <= breathOff) breathingActive = false;
```

Der vollständige Quellcode FlameController.cs liegt kommentiert im Git-Repository.

5.6 Herausforderungen und Lösungen

Während der Umsetzung traten mehrere typische Probleme auf, deren Lösung für das Verständnis des Projekts für mich besonders lehrreich war. Anfangs zerfiel die Flamme zu wilden „Lichtstrahlen“, weil das Skript versehentlich alle Unter-Partikelsysteme steuerte, auch Funken und Asche, deren erhöhte Geschwindigkeit die Partikel weit herauschoss. Die Lösung war, gezielt nur die beiden Kern-Systeme flames und flames secondary über ein Inspector-Feld anzusteuern und die Geschwindigkeit unangetastet zu lassen. Anschließend wurde die Flamme beim Start plötzlich dünner und kleiner, weil das Skript die Multiplikatoren des Assets mit einem Wert nahe 1,0 überschrieb. Gelöst wurde das, indem die Ausgangswerte in der Start-Methode gemerkt und der Puls-Faktor darauf multipliziert wird. Danach war der Effekt zunächst kaum sichtbar, weil der Wertebereich zu eng gewählt war; durch das Spreizen der Bereiche - Emission bis zum 4,5-Fachen, Größe vom 0,7- bis 2,4-Fachen - und die Farbe als zusätzlichen Kanal wurde der Unterschied deutlich.

5.7 Begründung zentraler Entscheidungen

Mehrere Entscheidungen musste ich am Anfang des Projekts berücksichtigen. VR wurde gegenüber AR gewählt, weil es eine kontrollierte, ablenkungsfreie Umgebung bietet, die zum Ruhe- und Fokus-Gedanken passt und von Unity gut unterstützt wird, während AR mit Tracking- und Lichtproblemen für einen Anfänger-Prototyp mehr Risiko als Nutzen brächte. Statt eines Sliders steuert nun eine automatische Simulation mit ergänzender Tastatur den Puls, weil ein selbst schwankender Puls lebendiger und glaubwürdiger wirkt und sich „ruhig“ und „gestresst“ in der Präsentation gezielt vorführen lassen.

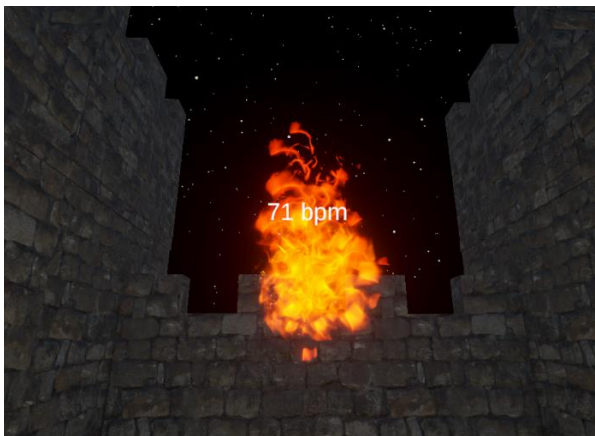
5.8 Future Work

Für eine Weiterentwicklung bietet sich zuerst die Anbindung eines echten Pulssensors über einen Raspberry Pi an,

der bpm-Werte per Netzwerk beziehungsweise JSON an die bereits vorbereitete zentrale Schnittstelle liefert. Weiter denkbar sind eine bpm-Verlaufskurve als kleines Diagramm in der Szene, ein optionaler CSV- oder JSON-Export des Puls-Verlaufs für Auswertungen sowie mehrere synchron reagierende Fackeln, um die Szene reicher wirken zu lassen.

5.9 Fazit

Im Ergebnis ist ein lauffähiger Prototyp entstanden, der eine simulierte Herzfrequenz auf mehreren Ebenen zugleich erfahrbar macht, nämlich über die Größe, die Fülle, das Flackern und die Farbe der Flamme, über ein mitreagierendes Licht, über einen hörbaren Herzschlag sowie über eine Atem-Anleitung. Die gewählte Metapher der Flamme erwies sich als gut verständlich, und der angestrebte Rückkopplungskreis, bei dem Anspannung sichtbar gemacht, zum Gegensteuern aufgefordert und die Veränderung anschließend dargestellt wird, funktioniert wie beabsichtigt. Als Person ohne vorherige Unity-Erfahrung konnte ich mir die Engine und die Programmiersprache C# im Verlauf des Projekts aneignen, wobei die Unterstützung durch die künstliche Intelligenz vor allem die Einarbeitung und die Fehlersuche beschleunigt hat.



Quellenverzeichnis

Flamme: <https://assetstore.unity.com/packages/vfx/particles/fire-explosions/free-fire-vfx-urp-266226>

Burg: <https://assetstore.unity.com/packages/3d/environments/fantasy/my-fantasy-fortress-402356>

Sterne: <https://assetstore.unity.com/packages/3d/environments/sci-fi/real-stars-skybox-lite-116333>

Claude: claude.ai